EEL4746
Microcomputers I

Real-Time Gesture Recognition and WiFi Data Transmission Using Arduino Nano 33 BLE Sense and ESP8266

Team Members

Name: **Samantha Scott** PID: **6355064**

Name: **Andrew Mendez** PID: **6463899**

Name: **Sharnece Young** PID: **3355710**

Name: **Ryan Berish** PID: **6424580**

Florida International University

Project Objectives:

The objective of our final project was to design and implement a real-time hand gesture recognition system using the Arduino Nano 33 BLE Sense and Edge Impulse. The data captured from the onboard gyroscope and accelerometer of the microcontroller made it possible for us to classify hand gestures such as: up-down, left-right,circle and idle. This data was then transmitted wirelessly through the NodeMCU ESP8266 Wifi Module and then sent to a web server so that we could remotely monitor and view our hand gesture data in real-time on a website.

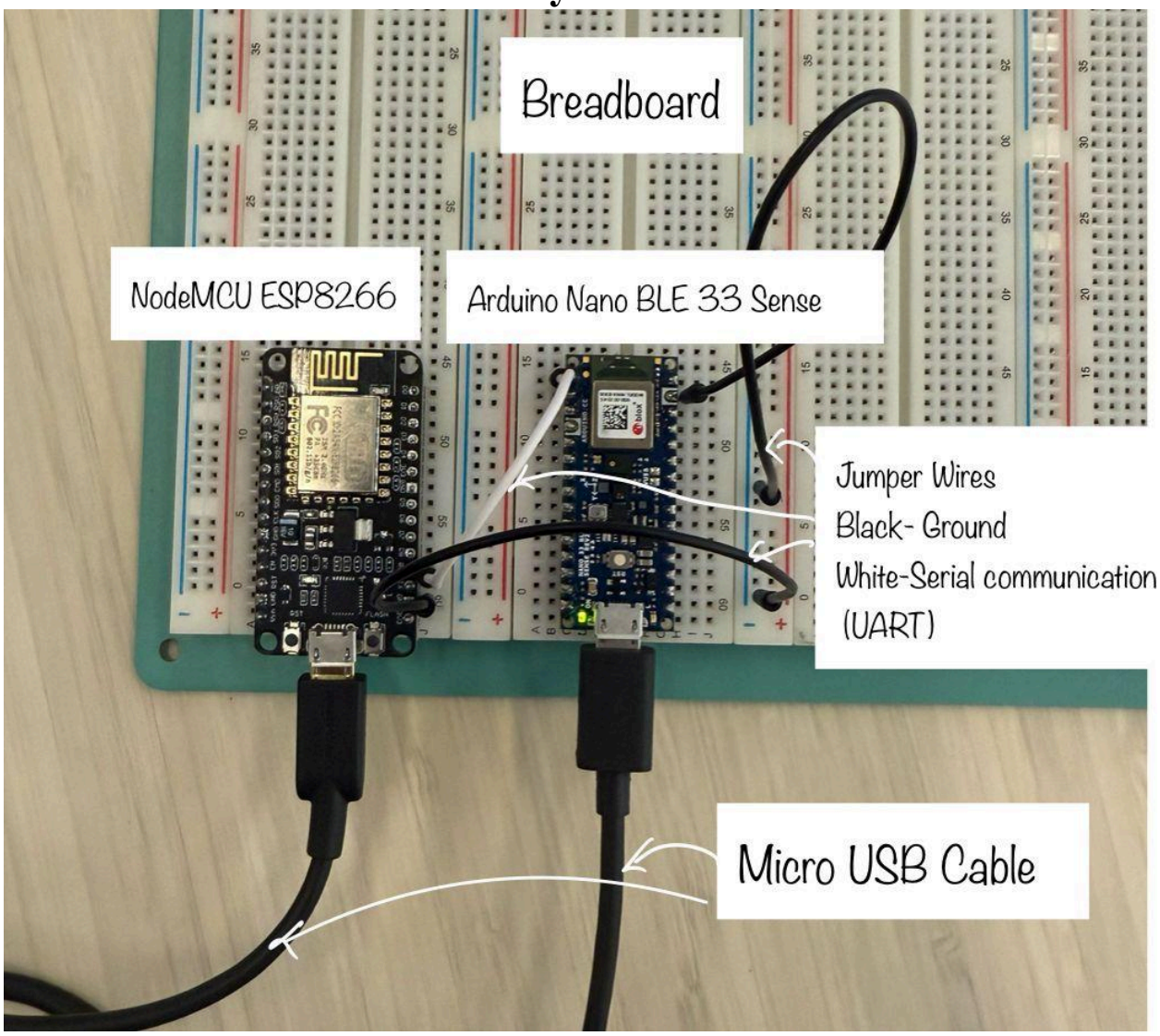
Our main objectives were:

- To use Edge Impulse to successfully collect,train, and deploy a TinyML model to our Arduino Nano 33 BLE.
- To enable IoT communication by integrating our Arduino with a secondary WiFi module (NodeMCU ESP8266)
- To display detected gestures in real-time on a simple website.
- Create a complete IoT pipeline:
  Arduino sensors → ML inference → WiFi → cloud/web dashboard

Components used

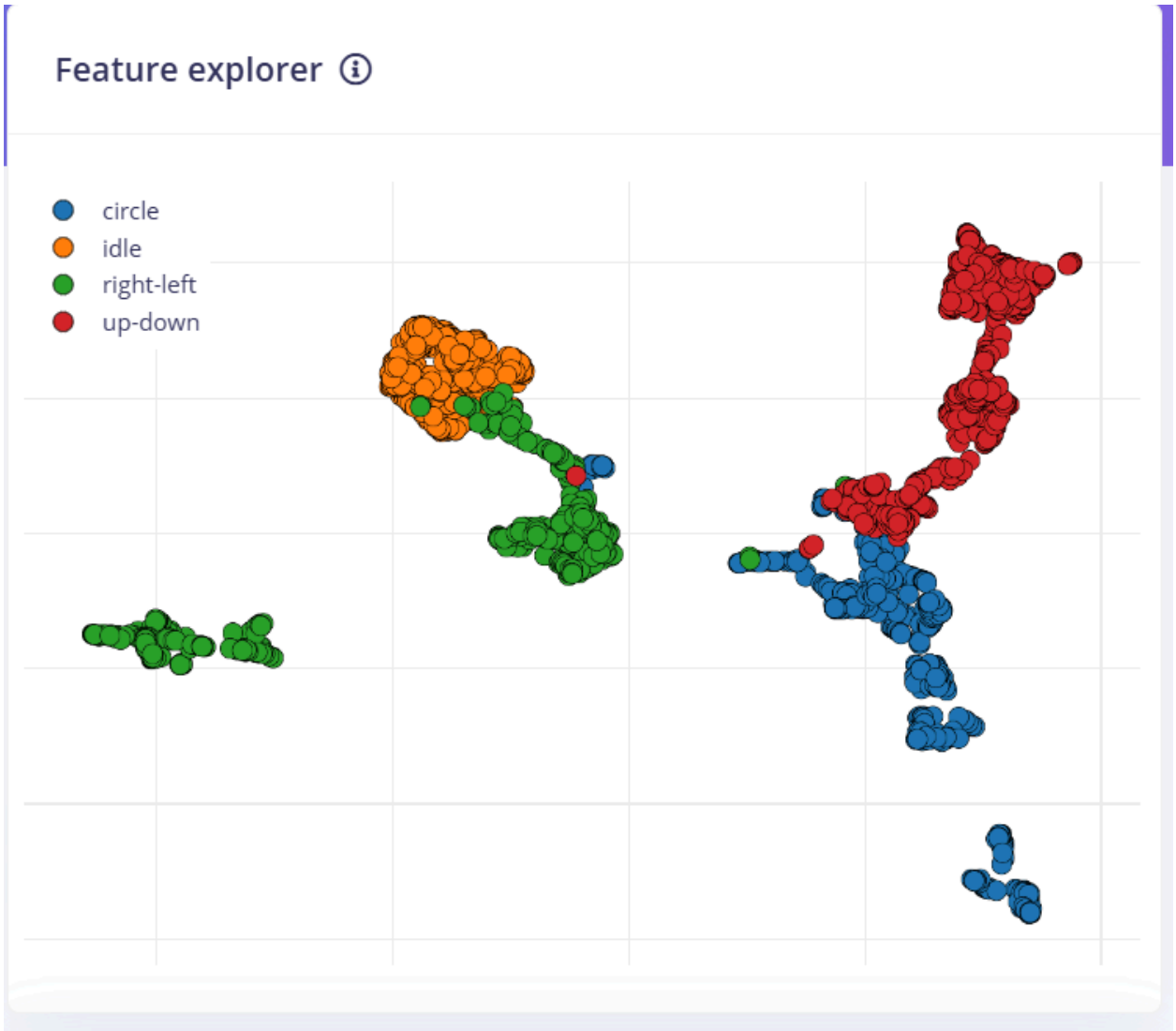
Component	**Purpose**
Arduino Nano 33 BLE Sense Rev2	Main microcontroller unit with built in gyroscope and accelerometer for gesture data collection and inference
NodeMCU ESP 8266	Wifi module to send gesture output to a local network
Micro USB Cable x 2	To power and program the Arduino and WiFi Module.
Jumper Wires	To connect Arduino and ESP8266 to enable serial communication between them (UART)
Breadboard	To assemble circuitry
Arduino IDE	To code and upload firmware to the microcontroller To read program output on the serial monitor
Edge Impulse Studio	Used for data collection, training and deployment of our Machine learning model.

Picture or Schematic of the System

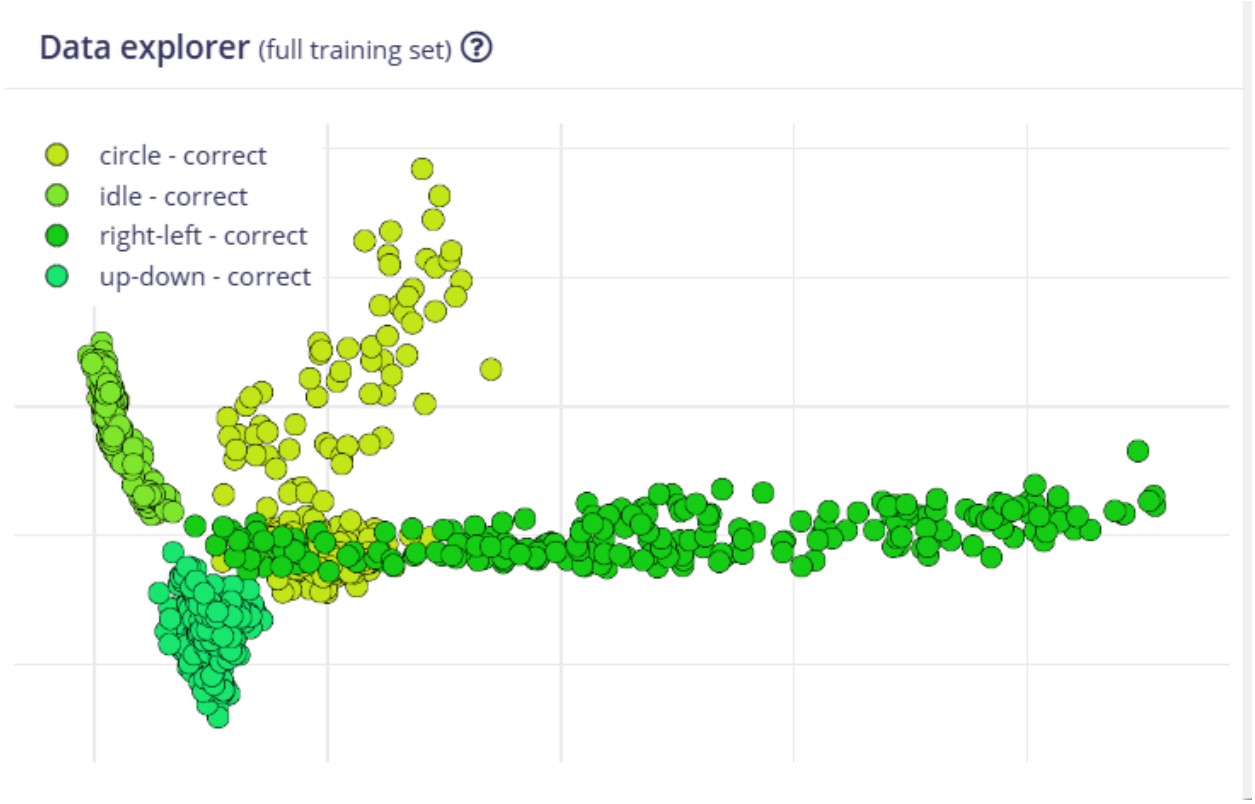


Edge Impulse Screen Shots

1. Classified features after feature extraction in edge impulse



2. Classified features after the model is trained in edge impulse



Software

Arduino Code

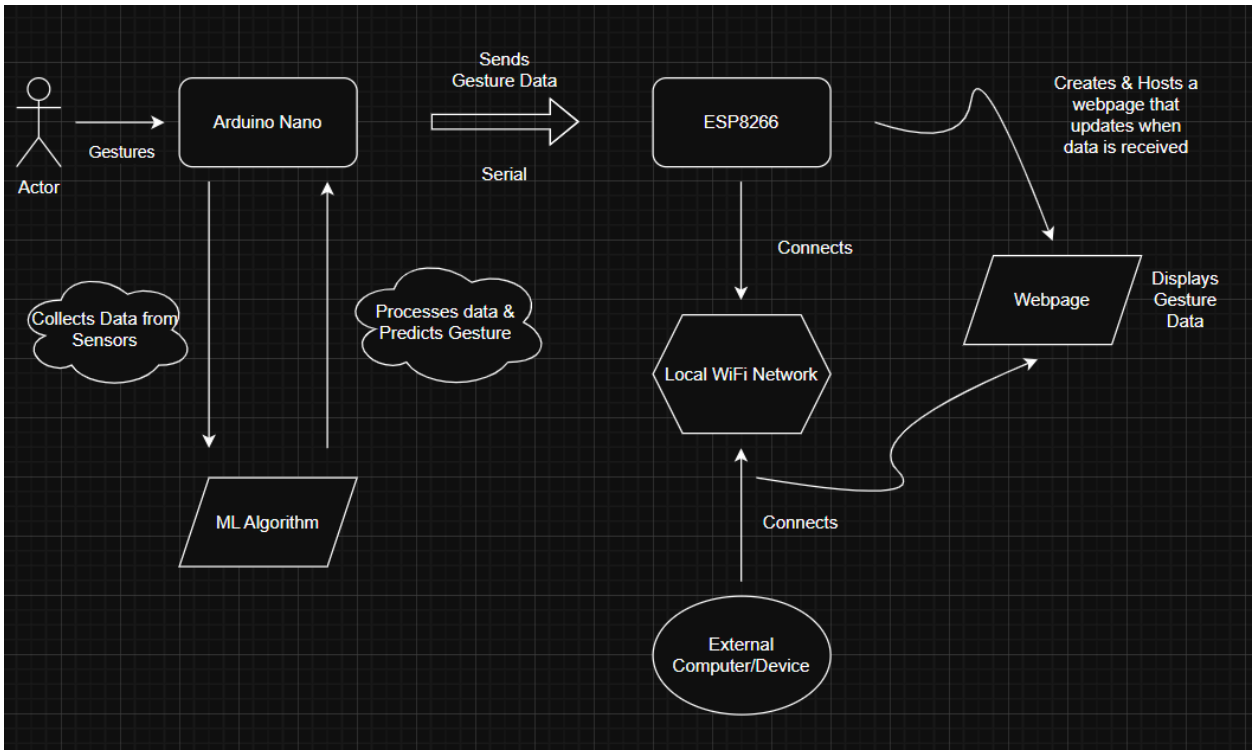
Library zip
https://drive.google.com/file/d/1SIkabsD5HiR60McxNUsUfbcAC32Uo8mN/view?usp=sharing

.ino file
https://drive.google.com/file/d/1IDrIJ9EjKOlWaChH7IDIvoRSI1CkEOZ0/view?usp=sharing

ESP8266 Code

.ino file:
https://drive.google.com/file/d/1s7-3BAb9GV8RANVq9GkJ-neAV5BQu6rA/view?usp=sharing

Project Flowchart



Troubleshooting

We first tried to connect the NodeMCU ESP8266 module with Florida International University's secure Wi-Fi network (FIU_SECUREWiFi) when setting up the system. The ESP8266 does not natively support enterprise-level authentication protocols (e.g., WPA2-Enterprise) required by FIU's network architecture. Instead, the ESP8266 supports WPA2-PSK, which uses a more basic pre-shared key authentication method. Due to this discrepancy in the protocols, the microcontroller did not authenticate with the university's secure wireless infrastructure. This incompatibility led to a significant limitation in providing for real-time debugging or testing in the classroom. Without a Wi-Fi supporting local system network, the ESP8266 couldn't send classification results to a dashboard or be notified of OTA (Over-the-Air) updates.

So due to the low cellular coverage in the classroom. The smartphone hotspot was one solution, but due to poor reception, connections were unstable or slow. In order to have a dependable testing ecosystem, we instead used a laptop-based hotspot feature, which was activated through the building's reduced reception but acted as a better local access point, but that technique also had the drawback of being restrictive. The laptop hotspot had very limited bandwidth capacity, supporting only a maximum of eight simultaneous connections. While the configuration worked, it proved to be inappropriate for fast-paced iterative testing or high-frequency data transmission, and for larger environments.

Upon model training, Edge Impulse presented optimal overall output when aggregating the data, namely at the time of feature development and initial classifier checks. However in real-world application, the Arduino Nano 33 BLE Sense Rev2 performance of the model dropped sharply upon deployment. The main problem lay in the uniformity of the training data. Gestures were sampled at a similar frequency and executed with similar range. Resulting in a relatively small spread of accelerometer and gyroscope values, which made the model overfit to one gesture processing style. As a result, the model had trouble generalizing when users performed gesture behavior slightly differently: smaller, larger, slower, more rapid than what was trained. To remedy this limitation, we expanded our database to represent broad scope of gesture variations:

- Small-amplitude and high-amplitude movements
- Slow, moderate, and rapid repetitions
- Variations in user technique and orientation
- Different sampling frequencies to improve generalization

This retraining process significantly improved the robustness of the classifier in everyday usage.

Circle action poses a particular challenge because the action of the circle gesture overlaps with parts of Up-and-Down gesture, particularly in the vertical acceleration axis. Consequently, the classifier frequently mixed up the two, leading to spikes in misclassification during circular movements. For a closer mapping between gesture signatures, we applied a low-pass filter at the preprocessing stage. This filter eliminated high frequency noise, smoothed the sudden sensor fluctuations, and distinguished the underlying motion path better. By preprocessing the data, the circular path became more continuous and periodic, while the Up-and-Down gesture remained sharp. The improvement of this method finally improved the distinction between each type of gesture.

Recommendations

Although our present system has only weak local network controls, a later version might be to launch either a remote website or be cloud based. A custom front-end would allow gesture classifications displayed in real-time, logs generated, IMU data visualized, and interactivity controls to debug. This would improve the project for polish, accessibility, and scalability beyond a local network.

The misclassification of circle and up-and-down movements indicates that some features (vertical acceleration specifically) need to be weighted differently. Through improving the

engineering of features or trying out some higher-end model, the system can pay more attention to particular gestures that differ.

The model could also be generalized to detect larger gestures, such as diagonal, wrist rotation, or multi-step sequences, extending usability of it. It would expand applications to include gaming interfaces, accessibility, and IoT control systems.

Conclusions

For this project, we successfully designed and implemented a real-time hand gesture system using the Arduino Nano 33 BLE Sense and Edge Impulse, which then transfers the data to a self-hosted website from a NodeMCU ESP8266. This serves as a building block for future projects that would need to have motion data wirelessly transmitted in real-time. This could be used for many applications, from simple tasks like powering devices on or off to remotely measuring motion data for medical research. While in the end we were able to get this project to work, that wasn’t without difficulties, like the FIU secure network and the circular motion data being confused for our Up-and-Down gesture. Due to this, the project does currently have some limitations with both the networking and motion data. Thankfully, those can be solved in future projects by hosting the website remotely rather than relying on the ESP8266 and training different movements so that the AI model won't get them confused.